

Section 3.1 — High-Availability Gateway Topologies

Enhanced Study Notes (Architecture-Heavy Section)

Study Directive: Give extra weight to before/after architecture diagrams and production topology comparisons between named systems (e.g., AWS NLB vs ALB, HAProxy vs Envoy, Nginx vs Traefik).

Executive Overview

High-availability systems eliminate single points of failure. Engineers must choose between active/passive and active/active configurations, understanding the trade-offs between cost, complexity, and resilience. This section covers the full spectrum from basic failover to multi-region active/active clusters, with production-grade considerations at each level.

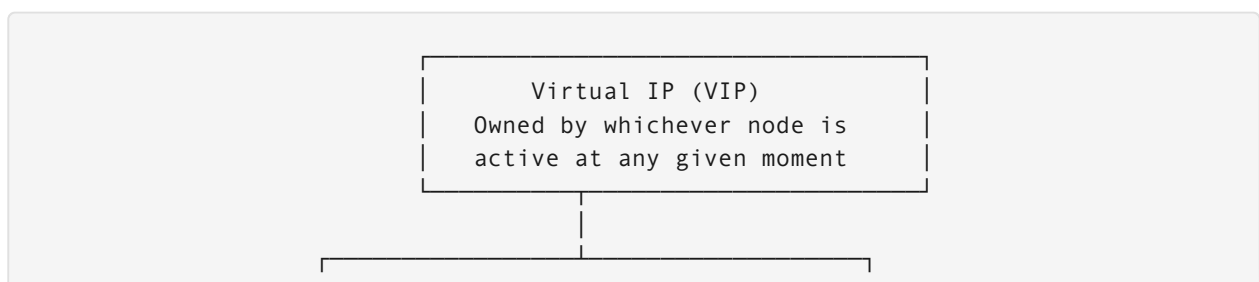
1. Ingress Resiliency Topologies

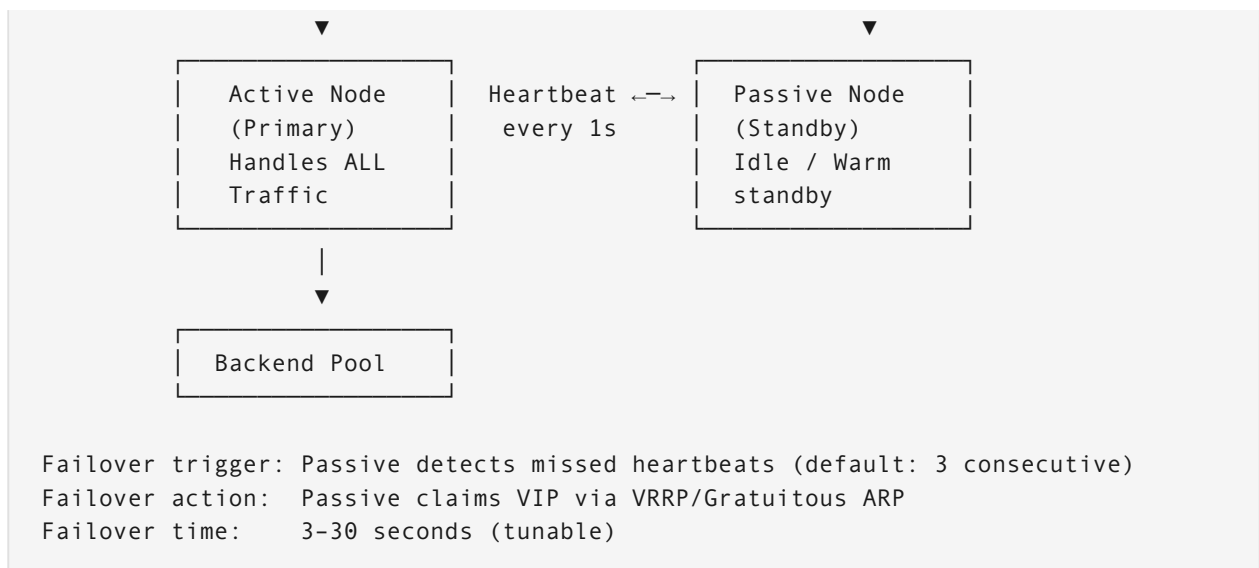
1.1 Active-Passive Architecture

Before (Single Node — SPOF):

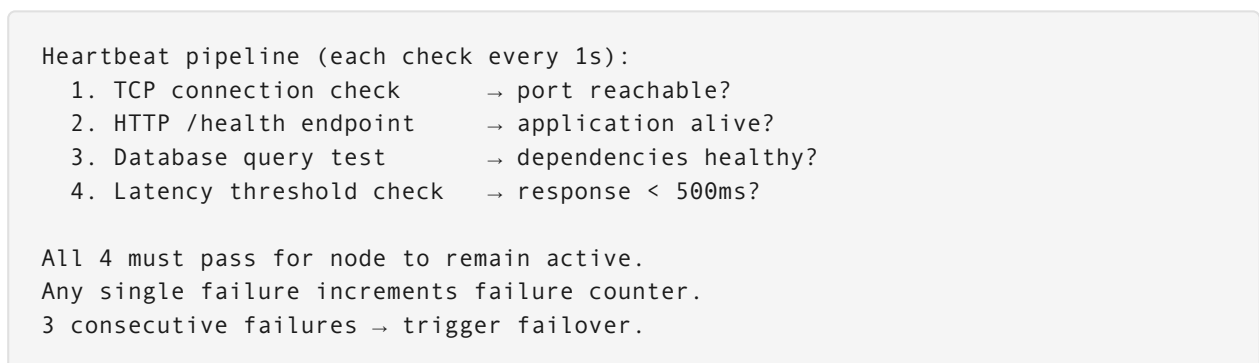
```
Internet → [Gateway Node] → Backend Services
           ↑
       Single point of failure:
       node dies = full outage
```

After (Active-Passive HA):



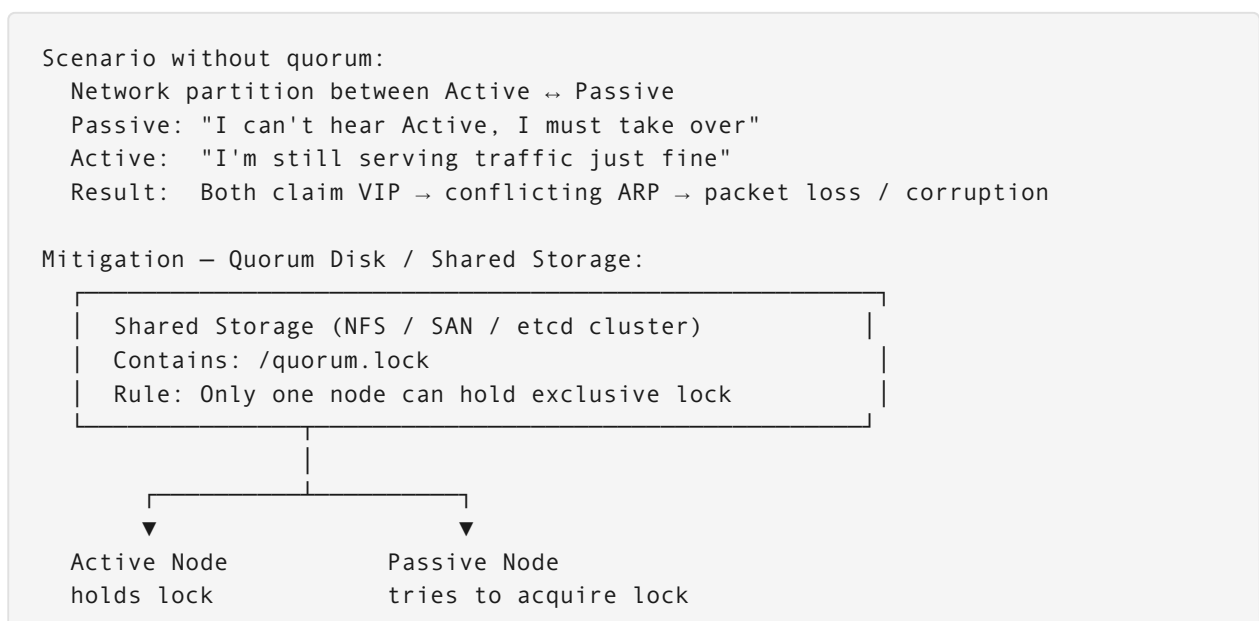


Health Check Mechanism:



Split-Brain Problem and Mitigation:

Split-brain occurs when both nodes believe they are active simultaneously (e.g., network partition between them, but both can still reach clients).

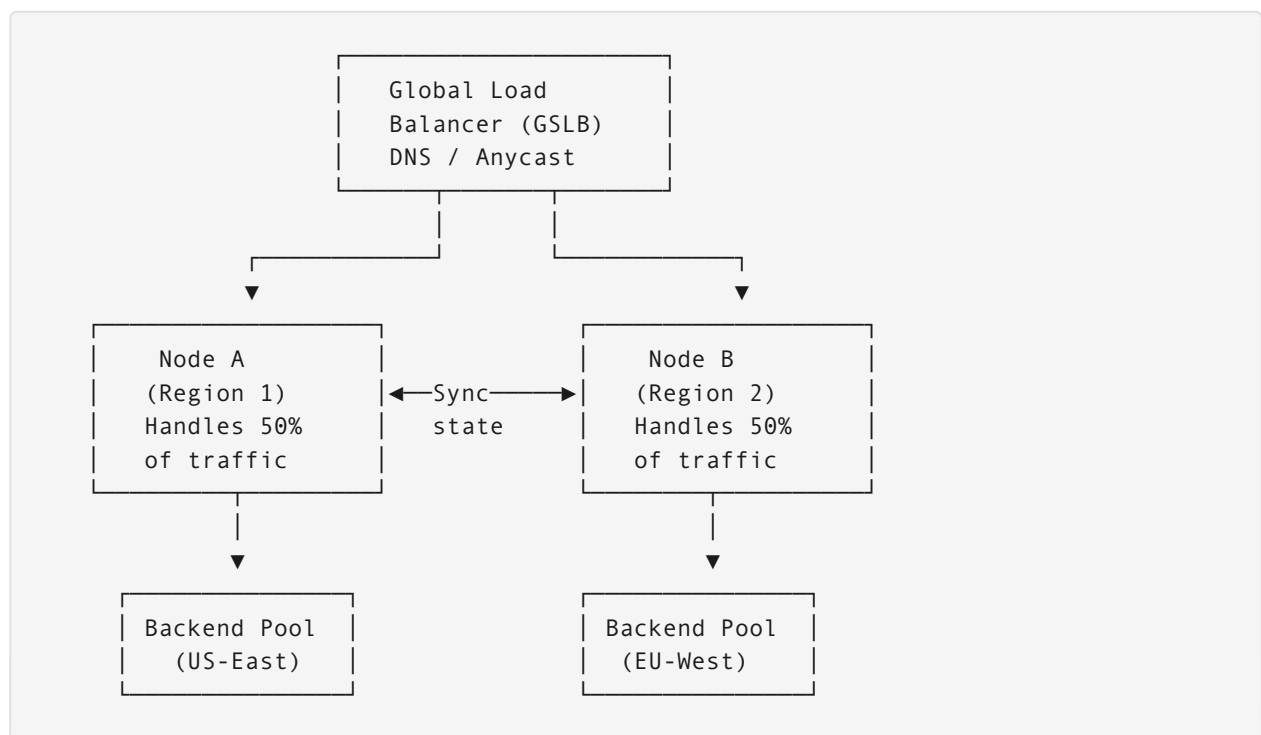


If Active dies → lock expires (TTL) → Passive acquires → promotes
If quorum storage unavailable → neither node promotes → manual intervention

System	Mechanism	Failover Time
HAProxy + Keepalived	VRRP	3–5s
PostgreSQL + Patroni	etcd quorum	10–30s
AWS RDS Multi-AZ	Synchronous replication	60–120s
Redis Sentinel	Raft leader election	30s

Before (Active-Passive — Wasted Capacity):

After (Active-Active — Full Utilization):



On node failure: GSLB detects via health checks,
routes 100% traffic to surviving node.
Surviving node must handle full load!
(Over-provisioning required: typically 60% capacity each)

Conflict Resolution — The Core Challenge:

Active-active introduces write conflicts when both nodes accept mutations simultaneously.

Stateful Sessions:

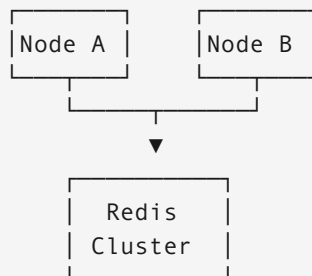
Problem: User authenticates on Node A.
Next request routed to Node B.
Node B has no session → 401 Unauthorized.

Solution A — Sticky Sessions:

GSLB hashes client IP → always routes to same node.
Risk: node failure breaks all its sessions.

Solution B — External Session Store:

Both nodes read/write to shared Redis cluster.
Any node can serve any session.
Adds ~1ms latency per request for Redis lookup.



Data Writes — Multi-Master Conflict:

Problem: Node A writes `user.balance = 100`.
Node B writes `user.balance = 200` simultaneously.
Both replicate. Which wins?

Solution A — Last-Write-Wins (LWW):

Use timestamps. Higher timestamp wins.
Risk: Clock skew (NTP drift $\pm 10\text{ms}$) causes data loss.

Solution B — CRDTs (Conflict-free Replicated Data Types):

Design data structures that merge deterministically.
Example: Counter CRDT always adds (never absolute set).
`user.balance = max(nodeA_balance, nodeB_balance)` ← wrong
`user.balance = sum(all_increments)` ← correct CRDT

Solution C — Coordinator / Leader Election:

One node is elected leader for writes.
Other nodes route writes to leader.

Raft / Paxos for leader election.
Reverts to a form of active-passive for writes only.

Named Systems Using Active-Active:

System	Sync Mechanism	Conflict Strategy
Cassandra	Gossip protocol	LWW with vector clocks
CockroachDB	Raft per range	Serializable transactions
AWS Route53	DNS TTL + health checks	Geographic routing
Cloudflare Load Balancer	Anycast BGP	Health-based steering

1.3 Trade-Off Matrix

Topology	Complexity	Monthly Cost	Availability	Recovery Time	Best For
Single Node	Very Low	\$100	99.5%	Minutes	Dev/test
Active-Passive	Low	\$200	99.9%	30s	Most production
Active-Active (2 nodes)	High	\$400	99.99%	<5s	High-traffic
Active-Active (3+ regions)	Very High	\$1200+	99.999%	<1s	Global services

2. Load Balancing Topologies

2.1 Layer 4 vs Layer 7 — Production Comparison

Before (L4 only — Limited Routing):

Client → [L4 LB: TCP port 443] → Any backend
↑

All HTTPS traffic goes to all backends.
Cannot route /api to API servers, /static to CDN proxy.

After (L7 — Content-Aware Routing):

```
Client → [L7 LB: HTTP inspection] ┌─→ /api/*           → API Backend Pool
                                   │→ /static/*         → CDN Proxy Pool
                                   │→ Host: admin.*       → Admin Backend Pool
                                   └─→ /health           → 200 OK (no backend hit)
```

Layer 4 Load Balancing — Deep Dive:

Transport Layer (IP + TCP/UDP):
Decision inputs: Source IP, Dest IP, Source Port, Dest Port, Protocol
No HTTP parsing: Bytes treated as opaque stream
Latency: Sub-millisecond (kernel-level with IPVS)
Memory per conn: ~100 bytes (just connection state)

Configuration (HAProxy L4):
frontend tcp_in
 bind *:443
 mode tcp
 default_backend https_servers

```
backend https_servers
  mode tcp
  balance least_connections
  option tcp-check          # Just: can I connect? No HTTP probe
  server s1 10.0.0.1:443 check
  server s2 10.0.0.2:443 check
```

Production systems: AWS NLB, F5 BIG-IP, Linux IPVS (LVS)
Throughput: 10M+ connections/second at 40Gbps (NLB)

Layer 7 Load Balancing — Deep Dive:

Application Layer (HTTP/HTTPS):
Decision inputs: Headers, URI, cookies, body, TLS SNI
HTTP parsing: Full request parsed before routing
Latency: 1-5ms additional overhead
Memory per conn: ~10KB (buffers + TLS state)

Configuration (NGINX L7):
upstream api_backend {
 least_conn;
 server 10.0.0.1:8080;
 server 10.0.0.2:8080;
}

```

server {
    listen 443 ssl;
    location /api/ {
        proxy_pass http://api_backend;
    }
    location /static/ {
        proxy_pass http://cdn_proxy;
        add_header Cache-Control "max-age=86400";
    }
}

```

Production systems: AWS ALB, NGINX, Envoy, Traefik, Caddy

Named System Comparison:

Feature	AWS NLB (L4)	AWS ALB (L7)	Envoy (L7)	HAProxy (Both)
WebSocket support	✓ Native	✓ Upgrade	✓	✓
gRPC routing	✓ (opaque)	✓ (aware)	✓ (native)	✓
Request-based routing	✗	✓	✓	✓
TLS termination	✓	✓	✓	✓
Throughput	40Gbps+	~10Gbps	~5Gbps/node	~100Gbps (kernel)
Latency overhead	<0.1ms	1–3ms	1–5ms	<0.5ms

3. Load Balancing Distribution Algorithms

3.1 Algorithm Comparison

Round Robin:

Nodes: [A, B, C]
 Requests: 1→A, 2→B, 3→C, 4→A, 5→B, 6→C ...

Problem — unequal capacity:
 A: 4 CPU cores (fast)
 B: 4 CPU cores (fast)
 C: 1 CPU core (slow)
 Round robin sends equal traffic → C becomes bottleneck
 Solution: Weighted Round Robin

Least Connections:

Nodes: A (10 conns) B (5 conns) C (3 conns)
Next request: → C (fewest)

Better than round robin for variable request durations.
Problem – tie-breaking: C ties with D at 3 conns each.
Solution: Use secondary sort (e.g., round-robin among ties).

Weighted Round Robin (Example):

Node capacities: A=4 CPU, B=2 CPU, C=2 CPU → weights 2:1:1

Sequence: A, B, C, A, A, B, C, A ...
 2 1 1 2

(Every 4 requests: A gets 2, B gets 1, C gets 1)

Configuration (NGINX):

```
upstream backend {  
    server 10.0.0.1 weight=2; # Gets 2x traffic  
    server 10.0.0.2 weight=1;  
    server 10.0.0.3 weight=1;  
}
```

Consistent Hashing:

Ring (0-100):

Virtual node positions:

A: 5, 50, 95
B: 25, 60, 90
C: 35, 70, 85

Request K=18 → hash(K)=18 → next clockwise = B@25

Request K=40 → hash(K)=40 → next clockwise = C@70 (wait, 40 → A@50)

Why virtual nodes?

Without VNodes (A@50, B@60, C@70):

Keys 0-50 → A (50% of ring)
Keys 51-60 → B (10% of ring)
Keys 61-70 → C (10% of ring)
Keys 71-100 → A (30% of ring) ← A handles 80%!

With VNodes (3 per node):

Each node handles ~1/3 of ring = balanced

4. Consistent Hashing Deep Dive

4.1 Rebalancing on Node Changes

Node Join — Minimal Key Movement:

```
BEFORE (nodes A, B, C):  
Ring: A@10, B@40, C@70  
Keys: K=5→A, K=20→B, K=50→C, K=80→A  
  
AFTER (add D@55):  
Ring: A@10, B@40, D@55, C@70  
Keys: K=5→A, K=20→B, K=50→D (MOVED), K=80→A  
  
Only keys between B@40 and D@55 moved.  
All other keys unchanged.  
Movement fraction = (new node range) / (ring size) = ~1/4
```

Node Leave — Recovery:

```
BEFORE (nodes A, B, C, D):  
Ring: A@10, B@40, D@55, C@70  
  
D crashes:  
Ring: A@10, B@40, ----, C@70  
Keys that were on D now route to C (next clockwise)  
Temporary overload on C until D recovers or keys migrate
```

4.2 Application Domains — Production Examples

Distributed Caching (Memcached at Facebook):

```
Problem: 1000 web servers → 100 Memcached nodes  
Naive modulo: key % 100 = node  
Node 5 added: key % 101 → almost ALL keys remap → cache cold!  
  
Consistent hashing:  
Node 5 added → only 1/101 ≈ 1% of keys remap  
99% cache hit rate preserved during scaling event  
  
Pre-warming new nodes:  
When D is added, B (its predecessor) has keys that now belong to D.  
Copy those keys to D before removing them from B.  
D serves requests immediately (warm cache).
```

Database Sharding (Real Example):

User IDs: 1M users across 4 shards

Shard mapping via consistent hash:

user_id=1001 → hash=0x3F2A → Shard2

user_id=1002 → hash=0x8B4C → Shard4

user_id=1003 → hash=0x1D7E → Shard1

Hot key problem:

Celebrity user has 100M followers.

user_id=9999 → Shard3 → all fan-feed queries hit Shard3.

Mitigation: Add random suffix to key: "9999_0", "9999_1" ... "9999_99"

Distributes across 100 virtual shards.

4.3 Algorithm Trade-Off Matrix

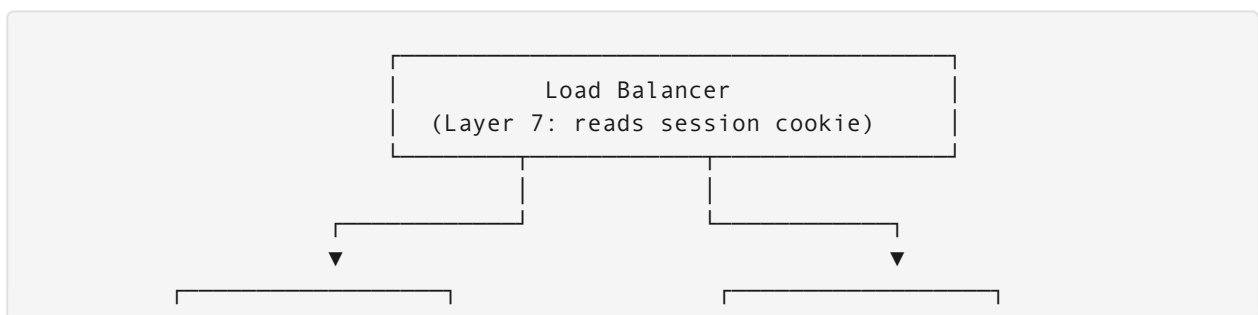
Algorithm	Load Balance	Rebalancing Cost	Complexity	Best Use Case
Modulo hash	Poor	$O(N)$ keys move	Very Low	Toy examples only
Consistent hash	Good	$O(K/N)$ keys move	Medium	Caches, CDN routing
Rendezvous hash	Excellent	$O(K/N)$ keys move	High	Fewer nodes, no VNodes needed
Jump hash	Excellent	$O(\log N)$	Low	Uniform, but no weighted nodes

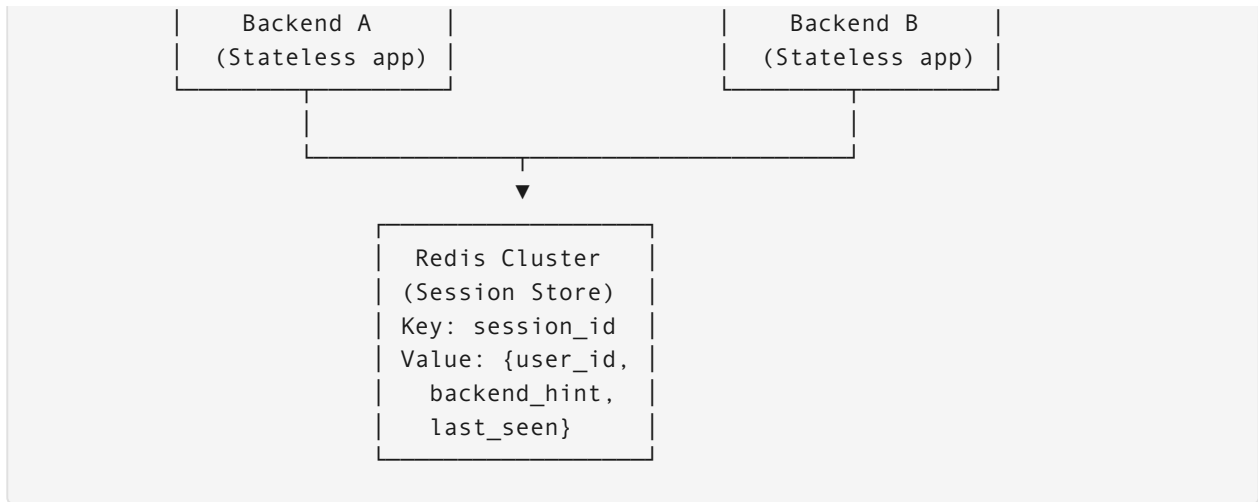
5. Interview Question — Extended Solution

Design a load balancer for a stateful service that needs session affinity. How do you handle node failures without dropping active sessions?

Solution Framework

Step 1 — Architecture Overview:





Step 2 — Session Routing Logic:

```
def select_backend(request):
    session_id = request.cookie.get('session_id')

    if session_id:
        # Check external store for preferred backend
        data = redis.get(f"session:{session_id}")
        if data:
            preferred = data['backend_hint']
            if is_healthy(preferred):
                # Refresh TTL on each use
                redis.expire(f"session:{session_id}", 3600)
                return preferred
            # Preferred backend down – fall through to reassignment

    # New session or backend failed: assign via consistent hash
    backend = consistent_hash(request.user_id or request.ip)
    new_session = generate_session_id()

    redis.setex(f"session:{new_session}", 3600, {
        'user_id': request.user_id,
        'backend_hint': backend,
        'created_at': time.now()
    })

    response.set_cookie('session_id', new_session, httponly=True, secure=True)
    return backend
```

Step 3 — Failure Scenarios:

Failure	Detection	Impact	Recovery
Hard crash	Health check timeout (~30s)	In-flight requests fail	Client retry → new backend assigned

Failure	Detection	Impact	Recovery
OOM / slowdown	Latency threshold breach	Slow responses	Remove from pool, drain gracefully
Planned maintenance	Manual signal	Zero impact	Drain: stop new sessions, wait for old to expire
Network partition	TCP timeout	Subset of users affected	GSLB reroutes

Drain Procedure (Planned Maintenance):

1. Signal backend: "entering drain mode"
 - Stops accepting new sessions
 - Returns 503 for new connections
 - Existing sessions still served
2. Load balancer: "stop routing new sessions here"
 - Marks backend as DRAINING (not DOWN)
 - Existing sessions still pinned
3. Wait: session TTL expires (5-30 minutes)
 - Monitor active connection count → 0
4. Take backend offline safely
 - Zero dropped sessions
 - Zero user impact

Key Insight: Perfect session preservation during failures requires shared session state. The trade-off is Redis overhead (~1ms/request) vs user experience continuity. For most services, Redis latency is acceptable. For ultra-low-latency services (<1ms budget), sticky routing only (accept session loss on failure).

6. Additional Interview Questions

Q2: Compare HAProxy vs Envoy for a microservices gateway. When do you choose each?

HAProxy:

Architecture: Event-driven, single-threaded per worker, C
 Strengths: Ultra-high throughput, battle-tested, simple config

Weaknesses: No native service discovery, limited observability
Best for: TCP load balancing, database proxying, traditional apps

Production topology:

Internet → HAProxy (L4/L7) → Microservice pods
Config reloaded via SIGUSR2 (no connection drops)
Used by: GitHub (8000+ RPM), Stack Overflow

Envoy:

Architecture: Event-driven, multi-threaded, C++, xDS API
Strengths: Dynamic config, rich observability (metrics/tracing), gRPC native
Weaknesses: Steeper learning curve, higher memory usage
Best for: Service mesh sidecars, dynamic microservices

Production topology:

Internet → Envoy (edge) → [Envoy sidecar → Service]
Config pushed via xDS server (no restarts)
Used by: Lyft (origin), Airbnb, Netflix

Decision Framework:

Choose HAProxy if:

- High-throughput TCP (databases, queues)
- Simple L4/L7 rules, rarely changing
- Team prefers simple config syntax

Choose Envoy if:

- gRPC or HTTP/2 microservices
- Need dynamic config (service discovery)
- Service mesh (Istio, Consul Connect)
- Need distributed tracing (Zipkin/Jaeger integration built-in)

Q3: A new deployment causes 10% of requests to fail. How does your load balancer detect and respond automatically?

Detection mechanisms:

1. Active health checks:
 - HTTP GET /health every 5s
 - Threshold: 2 consecutive failures → mark unhealthy
 - Threshold: 3 consecutive successes → mark healthy again
2. Passive health checks (circuit breaker):
 - Track error rate per backend in rolling 10s window
 - If error rate > 5% → eject backend for 30s
 - Progressive ejection: 30s → 60s → 120s (exponential backoff)
3. Outlier detection (Envoy):
consecutive_5xx: 3 # 3 x 5xx → eject

```
interval: 10s           # Check window
base_ejection_time: 30s  # Minimum ejection
max_ejection_percent: 50% # Never eject >50% of pool
```

Response:

- a. Remove unhealthy backend from pool
- b. Redistribute traffic to healthy backends
- c. Alert on-call (PagerDuty / OpsGenie)
- d. If all backends fail → return 503 with Retry-After header
- e. Canary rollback: if error spike correlates with recent deploy,
auto-revert to previous version (requires deployment pipeline integration)